

Automation of Index Creation in PostgreSQL

Team Members:

Lohit Adhitya (23b0952)
GnanaKoushik (23b1000)
Akshay Karthik (23b0981)
Maneendhar (23b0969)

<https://github.com/Gnanakousikreddy/auto-index-creation-postgresql-extension>

1 Motivation

Modern relational databases like PostgreSQL frequently suffer from performance degradation when tables lack appropriate indexes for heavily queried columns. Traditionally, Database Administrators (DBAs) must manually monitor slow query logs, analyze execution plans, and deploy indexes. This manual process is often reactive, labor-intensive, and prone to human error.

To solve this, our team developed the `auto_index` extension. The motivation was to transition PostgreSQL from a passive data store into a self-optimizing engine. By introducing an autonomous indexing system, the database can self-monitor sequential scans, perform mathematical cost-benefit analyses, and automatically deploy indexes when needed.

2 Functionality Implemented

The `auto_index` extension integrates directly into the PostgreSQL via its extensibility framework. The core functionalities include:

- **Query Interception:** Utilizing `ExecutorStart_hook` and `ExecutorEnd_hook` to intercept query execution plans and identify `SeqScan` (Sequential Scan) nodes.
- **Predicate Analysis:** Extracting the exact column attributes being filtered via equality predicates, separated by AND operator.
- **Hypothetical Costing:** Integrating with the **HypoPG** extension to generate virtual indexes and calculate the benefit of using the index without actually building the physical index first.
- **Global State Tracking:** Maintaining a shared memory Hash Table (HTAB) keyed by relation and attribute OIDs to track the frequency and cumulative benefit, across all concurrent database connections.
- **Autonomous Execution:** Deploying a non-blocking PostgreSQL Background Worker process to build the physical indexes concurrently once economic thresholds are met.

3 Code Created

The architecture is divided into four structural pillars:

- **`auto_index.c` (The Execution Engine):** It is the primary module. It initializes shared memory, sets up the executor hooks, intercepts queries, tracks the seqscans and interacts with HypoPG via SPI to calculate cost benefits.

- **auto_index_worker.c (The Background Worker):** A specialized background process is created at the start, which polls the shared memory state every 5 seconds to check if for any entries the threshold is crosses, and creates the index for those entries.
- **auto_index.h (Structural Definition):** Contains the shared memory layouts, such as `TrackingEntry` and `GlobalStats`.
- **auto_index-1.0.sql (User Interface):** Provides SQL-level abstractions, allowing users to query the system via custom functions, to observe the entries in the hash table and configure system behavior using GUC (Grand Unified Configuration) variables.

4 Benefit Calculation

To avoid frivolous indexing, the system evaluates the exact theoretical advantage of an index before building it. This is achieved by comparing the original query execution cost against a hypothetical optimized cost:

- **Cost Before:** The original execution cost is extracted directly from the active query execution plan. When a query is intercepted, the cost is read from the `queryDesc->plannedstmt->planTree->total_cost` structure provided by the PostgreSQL executor hook.
- **Cost After (Hypothetical):** To calculate the cost *as if* the index existed, the extension leverages **HypoPG** combined with internal PostgreSQL planner APIs.
 1. The system connects via the Server Programming Interface (SPI) and executes a `hypopg_create_index` command to create a virtual index for the candidate column in memory.
 2. Using `dlsym`, the extension dynamically loads the `hypopg_set_is_explain` function from the HypoPG library and sets it to `true`. This crucially signals HypoPG to actively expose the hypothetical index to the planner.
 3. The original query string (`queryDesc->sourceText`) is then manually fed back through the internal PostgreSQL parser, analyzer, and planner (`pg_parse_query`, `pg_analyze_and_rewrite`, `pg_plan_queries`).
 4. The new, optimized cost is extracted from the newly generated `PlannedStmt->planTree->total_cost` struct.
 5. Finally, a `hypopg_reset()` command is issued, and `set_is_explain` is reverted to `false` to ensure no residual state affects subsequent queries.

The difference between the original cost and the newly planned hypothetical cost represents the **Estimated Benefit** per execution. Because database queries are highly repetitive, this individual benefit is continuously aggregated in a shared memory hash table over time:

$$\text{Accumulated Benefit} = \sum (\text{Benefit}_i \times \text{Frequency}_i)$$

5 Criteria to Index

The decision to physically build an index is governed by the following condition :

Threshold: An index consumes disk space and incurs write overhead. The system calculates the one-time **Index Build Cost** by modeling the physical disk I/O and CPU cycles required to read table pages and sort tuples:

$$\text{Build Cost} = (\text{Blocks} \times \text{seq_page_cost}) + (\text{Tuples} \times \log_2(\text{Tuples} + 1.0) \times \text{cpu_operator_cost})$$

The physical index creation is triggered only when the accumulated benefit exceeds this build cost:

$$\text{Accumulated Benefit} > \text{Build Cost}$$

6 Index Creation

Once the threshold is crossed, the physical index must be created without blocking user transactions. This is handled autonomously by a dedicated **Background Worker** process, spawned by the PostgreSQL Postmaster.

- The worker attaches to the shared memory segment, reads the threshold flags, and establishes a connection to the target database.
- It constructs and executes a `CREATE INDEX CONCURRENTLY` SQL command.
- The `CONCURRENTLY` keyword ensures no `AccessExclusiveLocks` are taken. This allows the database to serve standard `SELECT`, `INSERT`, and `UPDATE` traffic completely uninterrupted while the new index is built in the background.

7 Testing

To validate the thresholds of the `auto_index` extension and the index creation, we conducted targeted tests using a standard equality predicate (`SELECT * FROM table_name WHERE column_name = value`). The tests were executed across three distinct data distributions to observe the cost-benefit calculations and index creation:

- **100,000 Rows (Random Values):** The target table was populated with 100,000 randomly generated values. The system accurately calculated a substantial *Estimated Benefit* per execution. Once the *Accumulated Benefit* surpassed the calculated *Build Cost*, we successfully observed the background worker autonomously creating the physical index.
- **1,000 Rows (Random Values):** For a smaller dataset of 1,000 random values, the system still recognized a positive benefit for index usage, smaller per query. After sufficient query repetitions pushed the accumulated benefit past the Build Cost, the index was correctly created as expected.
- **1,000 Rows (Uniform Values):** The table was populated with 1,000 rows where every value in the target column was identical. Because an index provides no selectivity advantage when all values match the query predicate, the HypoPG integration correctly estimated the hypothetical cost savings to be exactly zero. Consequently, the *Accumulated Benefit* remained at 0, and the system intelligently bypassed index creation. This successfully demonstrated that indexing on low-cardinality data is not done.

8 Use of AI in Development

- **Architecture & Data Structures:** Used it to understand dependencies between PostgreSQL files and understanding nested internal structures (e.g., `QueryDesc`, `PlannedStmt`).
- **HypoPG Integration:** Used it to install the extension, and to understand its usage in creating hypothetical indexes, which is used during benefit calculation.
- **Concurrency Management:** Used it to understand postgresQL-specific synchronization primitives (`LWLocks`, `Latches`) to safely manage shared memory.

- **Hooking Mechanisms:** Used it to understand how to intercept the query execution pipeline via `ExecutorStart_hook` and `ExecutorEnd_hook`.
- **Internal SQL Execution (SPI):** Used it to understand how to Utilize the Server Programming Interface to execute internal SQL commands .